

Introducción a Qiskit mediante la creación de fractales

Laboratorio No.1 de Computación Cuántica.

Jose Andres Henao Alzate
*Ing. Electrónica y
Telecomunicaciones*
Universidad de Antioquia
Medellín, Colombia
andres.henao9@udea.edu.co

Santiago Giraldo Tabares
*Ing. Electrónica y
Telecomunicaciones*
Universidad de Antioquia
Medellín, Colombia
santiagogiraldot@udea.edu.co

Jakob Kuhn
*Ing. Electrónica y
Telecomunicaciones*
Universidad de Antioquia
Medellín, Colombia
kuhn.jakob@mail.de

Abstract—

Este informe explora la relación entre la computación cuántica y la generación de fractales, basándose en el artículo “Creating Fractal Art With Qiskit” de Wiktor Mazin (2022). Se analiza cómo los circuitos cuánticos pueden aprovechar fenómenos como la superposición y el entrelazamiento para generar estructuras fractales. Las principales contribuciones del autor incluyen la implementación de un enfoque para la creación de fractales utilizando Qiskit y la publicación de Jupyter Notebooks que permiten la exploración interactiva de fractales generados cuánticamente. Se examina el funcionamiento general de estos notebooks, destacando su capacidad para visualizar fractales y modificar parámetros para alterar los resultados. Además, el informe presenta un análisis detallado de los circuitos cuánticos simulados, interpretando los datos obtenidos y evaluando su impacto en la computación visual cuántica. Se incluye una versión personalizada y documentada del código, junto con una discusión sobre los fractales generados y su comportamiento computacional. Finalmente, se presentan las conclusiones sobre la relevancia de la computación cuántica en la generación de fractales, acompañadas de las referencias bibliográficas utilizadas. **Index Terms—Reglas, Programación, Intertelco, indentación, Lógica inversa.**

Circuito cuántico, Entrelazamiento, Superposición, Fractal, Computación cuántica.

I. INTRODUCCIÓN

La computación cuántica ha demostrado su potencial en diversas áreas, desde la criptografía hasta la optimización de problemas complejos. Una de sus aplicaciones menos exploradas, pero de gran interés, es la generación de estructuras fractales. Los fractales son patrones autosimilares que se repiten en distintas escalas y aparecen tanto en la naturaleza como en modelos matemáticos. Mientras que los métodos tradicionales de generación de fractales se basan en algoritmos deterministas, la computación cuántica introduce nuevas posibilidades al aprovechar principios como la superposición y el entrelazamiento para producir variaciones únicas y altamente eficientes de estos patrones.

En este contexto, el artículo “Creating Fractal Art With Qiskit” presenta un enfoque novedoso para la generación de fractales utilizando circuitos cuánticos en la plataforma Qiskit.

En este informe, se analiza la relación entre la computación cuántica y la generación de fractales, se examinan las principales contribuciones del autor y se describe el funcionamiento de los notebooks disponibles en GitHub. Asimismo, se presentan los resultados obtenidos a partir de simulaciones de circuitos cuánticos, se interpreta su impacto y se documenta una versión modificada del código original. Finalmente, se exponen las conclusiones generales de la práctica, resaltando la importancia de la computación cuántica en este campo emergente.

II. MARCO TEÓRICO

A. Resumen del artículo

Los fractales, estructuras visuales basadas en la repetición de patrones a diferentes escalas, se conectan con la computación cuántica a través del uso de números complejos, esenciales para describir los estados de los qubits. En este contexto, el autor plantea que es posible generar fractales, como los conjuntos de Julia, a partir de los estados cuánticos obtenidos de circuitos diseñados en simuladores o computadoras cuánticas reales.

La idea central consiste en usar las amplitudes complejas del estado cuántico y combinarlas, por ejemplo, mediante el cociente entre dos de ellas, para definir el número complejo c utilizado en la fórmula del conjunto de Julia. Esta representación permite visualizar de forma artística y matemática la naturaleza probabilística y compleja de la información cuántica.

Entre sus principales contribuciones, el autor desarrolla un enfoque novedoso que une arte y ciencia, mostrando cómo la variabilidad inherente a las mediciones cuánticas produce fractales únicos en cada ejecución. También rinde homenaje a su profesor Erik Mosekilde, quien lo introdujo al estudio del caos, la dinámica no lineal y los fractales.

Además, el autor pone a disposición del público dos Notebooks en GitHub. El primero genera fractales a partir de un solo qubit, usando su estado final para calcular el fractal. El segundo trabaja con dos qubits entrelazados y permite observar fractales más complejos.

Ambos Notebooks usan Qiskit y el Circuit Composer, permitiendo experimentar con diferentes compuertas cuánticas para obtener patrones variados y estéticamente llamativos.

III. GENERACIÓN DE FRACTALES A PARTIR DE UN SOLO QUBIT

A. código en Python con Qiskit 2.0.0

A partir de la publicación de Mazin, la cuál incluía un código proof-of-concept en Python, empleando las librerías de Qiskit, el profesor Gustavo Patiño Álvarez [] realizó una migración de este a la versión 1.3 de Qiskit, dado que tras la actualización del SDK a la versión 1.0.0, todo código anterior a esta versión sería inutilizable. Cabe destacar que el citado profesor se valió de la guía de migración proporcionada por la propia documentación de Qiskit. Posterior a esto, se corroboró que dicho código actualizado a Qiskit 1.3 fuera plenamente compatible con la más reciente Qiskit 2.0.0, lanzada a la fecha de elaboración de este informe.

A continuación, se presentará una explicación detallada de cada bloque de código del notebook en cuestión. Para esta labor se toma en cuenta la documentación más reciente de Qiskit.

```
1 # Importing standard python libraries
2 %matplotlib inline
3
4 import numpy as np
5 from math import pi, sqrt
6 import matplotlib.pyplot as plt
```

El primer bloque de código, como suele ser práctica usual en la elaboración de scripts de Python, no es más que la importación de las librerías necesarias, que contienen las funciones que se emplearán en el programa: numpy: permite cálculos numéricos, así como creación de arreglos numéricos. np sería su abreviatura math: Incluye múltiples funciones y constantes matemáticas. En este caso, no es necesario importar todo, sino solo la constante pi y la función raíz cuadrada sqrt. Será necesaria para definir las funciones matemáticas como la función no-lineal sobre la que se iterará para generar el fractal. Matplotlib.pyplot: Permite la visualización de gráficos. Será de utilizada para graficar las distribuciones de probabilidad de los qubits. plt sería su abreviatura.

```
1 # Importing standard Qiskit libraries
2
3 from qiskit import QuantumCircuit,
4     QuantumRegister, ClassicalRegister
5 from qiskit import transpile
6 import qiskit
7
8 from qiskit.quantum_info import Statevector
9
10 from qiskit.visualization import
11     plot_distribution, plot_bloch_multivector
12 from qiskit.visualization import array_to_latex
13
14 from qiskit_aer import AerSimulator
```

Este bloque, continuando con la filosofía del anterior, incluye las instrucciones para importar los módulos de interés de la librería de Qiskit. Nótese que no se importa la totalidad

de funcionalidades, sino solo aquellas de interés. QuantumCircuit: Esta función permite definir un circuito cuántico. Sus argumentos son aquellos registros tanto clásicos como cuánticos que definamos. QuantumRegister: Esta función permite definir un registro cuántico. Sus argumentos incluyen principalmente el nombre del registro, y la cantidad de qubits en este. Definir un registro cuántico con esta instrucción es equivalente a insertar una nueva línea en el “pentagrama” del Quantum Composer de IBM. ClassicalRegister: Similar al anterior, permite definir un registro clásico. Sus argumentos incluyen principalmente el nombre del registro y la cantidad de bits en este. Es este registro sobre el cuál se colapsará el registro cuántico, almacenando entonces un valor determinístico. También es el equivalente a insertar una nueva línea (de registro clásico) en el “pentagrama” del Quantum Composer de IBM. Transpile: Esta instrucción es necesaria para “reescribir” nuestro circuito cuántico en otra topología que sea compatible con un determinado Hardware cuántico. También es necesaria para optimizar circuitos cuánticos y así, poder reducir el efecto del ruido. Statevector: permite almacenar la representación vectorial de los estados cuánticos. Plot-distribution: función rápida para la visualización de la distribución de probabilidad Plot-bloch-multivector: Permitirá la visualización, en esfera de Bloch, de los vectores de estado. AerSimulator: Simulador de circuitos cuánticos proporcionado por Qiskit. (requiere instalación por separado mediante pip install qiskit-aer).

```
1 InputQ = QuantumRegister(1, name='InputQ')
2 Medida = ClassicalRegister(1, name='Medida')
3
4 # Start with an one qubit quantum circuit
5     ↳ yielding a nice fractal. Change the circuit
6     ↳ as you like.
7 circuit = QuantumCircuit(InputQ, Medida)
8 circuit.h(0)
9 circuit.u(pi/4, -pi/3, pi/8, 0)
```

Similar a cuándo se definen las entradas o registros cuánticos en el IBM Quantum Composer como líneas vacías de un pentagrama, debemos crear los registros haciendo uso de QuantumRegister y Classical Register. Se crean con especios de 1 qubit y 1 bit respectivamente, y con nombres InputQ y Medida. Ambos, homónimos a las variables donde se almacenan.

Ahora, teniendo las líneas del “pentagrama”, se procede con la conformación del circuito mediante la ubicación de compuertas cuánticas sobre dichas líneas. Primero se define nuestro circuito “circuit”, así como qué líneas (o dicho de forma técnica, qué registros) lo componen. Para esto, se usa la función QuantumCircuit, cuyos argumentos son los registros previamente definidos. Teniendo el circuito definido, se procede con la ubicación ordenadas de compuertas cuánticas. Con circuit.h(0) se define una compuerta Hadamard al qubit en la posición 0. Cabe recordar que esto lo pondrá en superposición equilibrada entre $|0\rangle$ y $|1\rangle$. Con circuit.u(pi/4, -pi/3, pi/8, 0) se aplica una compuerta U. Como esta compuerta permite la creación de cualquier compuerta mediante la rotación con tres ángulos de Euler, se deben especificar dichos ángulos en orden

θ, ϕ, λ . Como último argumento, especifica el registro cuántico donde se ubicaría la compuerta.

```
1 # View the circuit quantum state on the Bloch
  ↳ sphere
2 out_qubit = circuit
3
4 plot_bloch_multivector(out_qubit)
5 plt.show()
```

Se visualiza el estado cuántico en una esfera de Bloch

```
1 # Definición de simuladores requeridos
2 statevector_simulator =
  ↳ AerSimulator(method='statevector')
3 # circuit_simulator = AerSimulator()
```

Una vez elaborado el circuito, es de nuestro interés poder simularlo. Para esto, primero se debe configurar el simulador (AerSimulator). Se configura en una variable statevector-simulator. La función AerSimulator, para inicializar el simulador, recibe como argumento un método. El AerSimulator soporta múltiples métodos y opciones de configuración. Del método dependerá el formato de la salida de la simulación. En este caso, al especificar que utilice el método “statevector”, el AerSimulator hace una simulación que permite muestrear medidas de salida a partir de circuitos ideales, con todas las medidas al final del circuito. Pare simulaciones con ruido, cada vez que se ejecuta una tirada o “shot”, se elige una versión aleatoria del circuito con efectos de ruido según el modelo definido.

```
1 # Copia del circuito para obtener el vector de
  ↳ estado final resultante
2 circuit2SV = circuit.copy()
3 circuit2SV.save_statevector()
```

Se crea una copia del circuito llamada circuit2SV mediante el método .copy() del circuito definido. A la copia realizada se le saca su vector de estado mediante el método .save-statevector().

```
1 # Transpilación para posteriormente realizar la
  ↳ simulación del vector de estado resultante
2 circ_statevector = transpile(circuit2SV,
  ↳ statevector_simulator)
3
4 # Run the circuit with the state vector simulator
  ↳ to obtain a noise-free fractal.
5 # Ejecución y obtención del statevector
6 result =
  ↳ statevector_simulator.run(circ_statevector)
7 .result()
8 out_statevector =
  ↳ result.get_statevector(circ_statevector)
9
10 display(array_to_latex(Statevector(out_statevector),
  ↳ prefix="\\ket{out_{statevector}} = "))
```

Para estas alturas ya se tiene un circuito cuántico creado. Lo siguiente sería compilarlo, para posteriormente ejecutarlo en el simulador. Para compilar el circuito (o en este caso, su copia), se hace uso de la función transpile(). Como se explicó previamente, “transpilar” un circuito es optimizar y adaptar su topología a una compatible con el backend (hardware cuántico o simulador) a utilizar. En este caso, se especifican como

argumentos el circuito (en este caso circuit2SV) y el backend o simulador en el que lo ejecutaremos más adelante (en este caso, se configuró previamente el AerSimulator con método “statevector” en la variable statevector-simulator). Cabe aclarar que no son los únicos argumentos que puede recibir la función transpile(). Esta, permite que se configuren muchos más aspectos a tener en cuenta en la compilación del circuito (constraints) con argumentos como basis-gates, coupling-map, dt, entre otros (referirse a la documentación de la función). La función retorna un tipo de variable -CircuitT, que no es más que el circuito transpilado.

Posterior a la transpilación del circuito, este es ejecutado en el simulador. Para esto, se utiliza el simulador con el método .run(), especificándole qué circuito se debe ejecutar (en este caso, el transpilado, circ-statevector) y se accede al resultado con .result(). De dicho resultado, se puede almacenar solamente el vector de estado accediendo con .get-statevector() especificándole el circuito como argumento.

Dicho vector de estado puede ser visualizado en notación matemática mediante LaTeX. Para esto, se hace uso de la función array-to-latex() cuyo argumento sería el arreglo de interés (vector de estado de la salida).

```
1 # Probabilidades asociadas al vector de estado
  ↳ resultante
2 print(out_statevector.probabilities())
```

También, del vector de estado resultante, se pueden obtener las probabilidades. Para esto se accede a ellas mediante .probabilities(). Dichas son mostradas al usuario mediante un print(). Debe tenerse en cuenta que el primer valor del arreglo sería la probabilidad para $|0\rangle$, mientras que el otro valor sería la probabilidad para $|1\rangle$.

```
1 # Extract the first element of the state vector
  ↳ as z0 and the second element as z1.
2 z0 = out_statevector.data[0]
3 z1 = out_statevector.data[1]
4
5 print(z0, z1)
```

Una vez elaborado y ejecutado el circuito mediante el simulador, y una vez obtenidos los resultados de la simulación, ya se tiene la información necesaria para la generación de fractales a través de números complejos asociados a nuestro sistema cuántico.

Lo primero, sería extracción de los coeficientes del estado cuántico obtenido. Estos se almacenan en variables z_0 y z_1 . Estos son análogos a α y β . Estos coeficientes, números complejos, se imprimen en pantalla para el usuario.

```
1 # Goal: One complex number for the Julia set
  ↳ fractal.
2 if z1.real != 0 or z1.imag != 0:
3     z = z0/z1
4     print("Pre z= ", z)
5     z = round(z.real, 2) + round(z.imag, 2) * 1j
6 else:
7     z = 0
8
9 print("z= ", z)
10
```

```

11 # Define the size
12 size = 100
13 heightsize = size
14 widthsize = size

```

Como bien lo expresa el artículo escrito por Mazin, a partir de una función de estos operadores se puede construir una función tal que, iterando sobre esta, se pueda obtener el *Conjunto de Julia*. En particular, se tiene que, para generar dicho conjunto, se necesita actualizar el valor de z mediante la fórmula:

$$z = z^2 + c$$

donde c es una constante compleja dada por nuestros coeficientes.

Lo primero es inicializar z en un valor semilla derivado de los coeficientes obtenidos del `statevector` del circuito. Para esto, se define que:

$$z = \frac{z_0}{z_1}$$

Es necesario que tanto la parte real como la parte imaginaria de z_1 sean distintas de cero, con el fin de que la división sea posible. En caso contrario, se inicializa $z = 0$.

Como nota adicional, se puede usar la función `round()` para redondear o acotar el número de decimales en z . En este caso, se dejan dos unidades para ambas partes: real e imaginaria.

Adicionalmente, se declaran `size = 1000` y `heightsize`, `widthsize = size`. Con esta información se define la resolución de la imagen a generar. En este caso, la imagen tendrá 1000×1000 “píxeles”.

```

1 def julia_set(c=z, height=heightsize,
  ↪ width=widthsize, x=0, y=0, zoom=2,
  ↪ max_iterations=100):
2
3     # To make navigation easier we calculate
  ↪ these values
4     x_width = 1.5
5     y_height = 1.5*height/width #Cambiar 1.5 por
  ↪ x_width para que el aspect ratio sea
  ↪ constante si se cambia el factor de
  ↪ escala
6     x_from = x - x_width/zoom
7     x_to = x + x_width/zoom
8     y_from = y - y_height/zoom
9     y_to = y + y_height/zoom
10
11    # Here the actual algorithm starts and the z
  ↪ paramter is defined for the Julia set
  ↪ function
12    x = np.linspace(x_from, x_to,
  ↪ width).reshape((1, width))
13    y = np.linspace(y_from, y_to,
  ↪ height).reshape((height, 1))
14    z = x + 1j * y
15
16    # Initialize c to the complex number obtained
  ↪ from the quantum circuit
17    c = np.full(z.shape, c)
18
19    # To keep track in which iteration the point
  ↪ diverged
20    div_time = np.zeros(z.shape, dtype=int)
21

```

```

22    # To keep track on which points did not
  ↪ converge so far
23    m = np.full(c.shape, True, dtype=bool)
24
25    for i in range(max_iterations):
26        z[m] = z[m]**2 + c[m]
27        m[np.abs(z) > 2] = False
28        div_time[m] = i
29    return div_time

```

Ahora bien, para la generación del conjunto de Julia, se define una función que ha de ser ejecutada de forma reiterativa. Esta función, denominada `julia_set`, tomará como argumentos:

- **c**: se trata de la constante compleja. Se parte inicialmente con un $c = z$, que previamente habíamos calculado como z_0/z_1 .
- **height**: se refiere al alto de la imagen del fractal. En este caso, se define un valor por defecto `heightsize`.
- **width**: se refiere al ancho de la imagen del fractal. En este caso, se define un valor por defecto `widthsize`.
- **x y y**: definen un *offset* para la visualización del fractal. Por defecto, se inicializan en $(0, 0)$.
- **zoom**: permite obtener una vista más cercana o lejana del fractal generado. Por defecto, se deja en 1, lo cual se interpreta como una escala 1:1, es decir, sin *zoom*.
- **max_iterations**: controla cuántas veces se repite la iteración de z . Converja o no, pertenezca o no al conjunto de Julia, este será el número máximo de iteraciones para cada punto.

Esta función retornará el “tiempo” o número de iteraciones que tardó cada punto en salirse del límite definido (valor absoluto mayor o igual a 2). Este tiempo, para cada punto (x, y) del plano de visualización, puede interpretarse como una coordenada z cuya magnitud conforma un mapa de calor, como ocurre en los espectrogramas.

Entre las líneas 8 y 9 del bloque de código de la función, se definen la ventana y sus dimensiones a partir de la altura y anchura proporcionadas como argumentos, y se realiza un reescalado a partir del valor de `zoom`. Nótese que para este reescalado, se almacenan los valores mínimo y máximo de x y y entre los que se ha de iterar, guardándolos como `x_from`, `x_to`, `y_from`, `y_to`.

Posteriormente, entre las líneas 12 y 14, se crean los arreglos de x y y sobre los que se hallarán los múltiples valores de z . Para ello se hace uso de la función `np.linspace()`, cuyos argumentos, en orden, son: valor inicial, valor final y número de valores en el rango. Luego, el arreglo x se convierte en un vector fila y el arreglo y en un vector columna mediante la función `reshape()`. Finalmente, se define la matriz bidimensional z inicial como:

$$z = x + jy$$

Luego, entre las líneas 17 y 23, se realizan las siguientes inicializaciones de variables:

- Se inicializa c como una matriz 2D, con la misma forma que z , cuyas entradas son todas iguales a la constante compleja c .
- Se inicializa la matriz `div_time`, del mismo tamaño que z , con todas sus entradas en cero. Esta matriz se define como una matriz de enteros.
- Se crea una máscara m , que es una matriz booleana del mismo tamaño que c , con todas sus entradas en `True`. Esta máscara actúa como una bandera para saber qué puntos siguen en iteración.

Entre las líneas 25 y 29, se comienza a iterar sobre la matriz z . En particular:

- En la línea 26, se aplica la fórmula de iteración $z_{n+1} = z_n^2 + c$ únicamente sobre aquellos puntos cuya bandera esté en `True`.
- En la línea 27, se actualizan las banderas a `False` para aquellos puntos cuyo valor absoluto sea mayor o igual a 2.
- En la línea 28, se actualiza `div_time` para aquellos puntos que aún están en iteración (es decir, los que permanecen con bandera `True`).

El número total de iteraciones por punto se retorna en `div_time`.

```
1 # plot the Julia set fractal
2 plt.imshow(julia_set(), cmap='magma') # viridis',
  ↳ 'plasma', 'inferno', 'magma', 'cividis'
3 plt.colorbar()
4 plt.show()
```

En este bloque de código, simplemente se grafica el fractal ejecutando de forma implícita la función `julia_set`, ya que su retorno es un argumento válido para la función `plt.imshow()`.

Adicionalmente, se agrega como segundo argumento de esta última el tipo de mapa de color que se desea utilizar para la visualización. En este caso, se define "magma" como la paleta de colores utilizada.

Una vez realizado todo esto, se tiene una idea de más o menos como debe verse el fractal generado para el circuito cuántico simulado previamente. Dado que ese estado cuántico obtenido actúa como una semilla generadora para el fractal, podemos esperar que, si se ejecuta en HW real o en un simulador con un modelo de ruido más realista, el fractal generado diferirá del recién obtenido. Para corroborar esto. Se procede con la siguiente parte del Notebook, el cuál es la ejecución del circuito cuántico en un simulador con modelo de ruido.

```
1 from qiskit_ibm_runtime import SamplerV2 as
  ↳ Sampler, QiskitRuntimeService
2 # from qiskit.transpiler import
  ↳ generate_preset_pass_manager
3
4 # The fake provider module contains fake
  ↳ providers and fake backends classes.
5 # The fake backends are built to mimic the
  ↳ behaviors of IBM Quantum systems using system
  ↳ snapshots.
```

```
6 # This allows to simulate a noise model generated
  ↳ from system snapshots obtained in the past.
7 # Here we choose a noise model based on an old
  ↳ backend named ibmq_Bogota of 5 qubits.
8 from qiskit_ibm_runtime.fake_provider import
  ↳ FakeBogotaV2
```

Como al inicio del *notebook*, lo primero que se realiza es la importación de los módulos necesarios para la ejecución del modelo en un entorno ruidoso. Desde la librería `qiskit_ibm_runtime`, específicamente del módulo `fake_provider`, se importa el backend `FakeBogotaV2`.

Este backend simula el comportamiento de un sistema cuántico real de IBM, incluyendo su modelo de ruido estadístico. En particular, `FakeBogotaV2` se basa en un antiguo backend físico de 5 qubits, conocido como `ibmq_bogota`.

Una vez importada la función, se instancia el backend como `ibmq_backend` utilizando esta clase simulada.

```
1 service = QiskitRuntimeService()
2
3 #ibmq_backend =
  ↳ service.least_busy(operational=True,
  ↳ simulator=False, min_num_qubits=127)
4 ibmq_backend = FakeBogotaV2()
5
6 #print("The least busy quantum computer now is:",
  ↳ ibmq_backend.name)
7 print("Now the chosen quantum computer is:",
  ↳ ibmq_backend.name)
```

`QiskitRuntimeService` es el servicio de acceso a los recursos de computación cuántica de IBM. Siempre que se desee acceder a servicios de hardware real o a simuladores, como los empleados en esta sección del *notebook*, es necesario crear una instancia de dicho servicio. En este caso, la instancia se nombra `service`.

Nota: Para acceder al hardware real, es necesario contar con un *token* de autenticación. En cambio, si se utilizan recursos simulados, como en este caso, se accede a través de un *fake provider*.

Una vez creado el servicio, se conecta a un backend simulado —es decir, no al backend de hardware real—. El backend utilizado es `FakeBogotaV2`, el cual simula la topología, cantidad de qubits, errores y modelo de ruido asociados al procesador IBM Bogota.

```
1 # Add measurement - used to calculate the
  ↳ modified state vector
2 circuit.barrier()
3 circuit.measure(InputQ, Medida)
4
5 circuit.draw('mpl', style="iqp")
6 plt.show()
```

El bloque de código anterior muestra cómo al circuito previamente creado para el simulador Aer se le agregan algunos elementos adicionales. Con `circuit.barrier()` se incorpora una barrera al final del circuito cuántico. Esta tiene la función de evitar que el transpilador reorganice o combine compuertas ubicadas a ambos lados de la barrera.

No afecta el estado cuántico del sistema, y además es común utilizarla como separador visual en el diseño del circuito.

Posterior a la barrera, se incluye un operador de medición mediante `circuit.measure()`. Esta operación tiene como objetivo colapsar el estado cuántico en un registro clásico. Los argumentos de la función corresponden al registro cuántico `InputQ` y al registro clásico `Medida`, respectivamente.

El circuito, tal como puede visualizarse en IBM Quantum Composer, también puede ser representado gráficamente desde el SDK mediante la instrucción `circuit.draw()`. El primer argumento indica que se debe utilizar `Matplotlib` como backend de visualización, mientras que los argumentos adicionales permiten ajustar el estilo del dibujo.

```
1 # Transpile the ideal circuit to a circuit that
  ↳ can be
2 # directly executed by the backend
3 transpiled_circuit = transpile(circuit,
  ↳ ibmq_backend)
4
5 transpiled_circuit.draw('mpl', style="iqp")
6 plt.show()
```

De forma similar a lo realizado con el `AerSimulator`, se trasquila el circuito cuántico (el mismo diseñado previamente) y se almacena en la variable `transpiled_circuit`.

Finalmente se vuelve a graficar el circuito, ahora transpilado, para observar qué transformaciones topológicas sufrió en dicho proceso.

```
1 sampler2run = Sampler(mode=ibmq_backend)
2
3 # Define the number of shots for the execution.
4 shotno=1024
5
6 # Run the circuit on a real quantum computer. NB!
  ↳ This may take a while.
7 job = sampler2run.run([transpiled_circuit],
  ↳ shots= shotno)
8
9 # ID del Job para rastrear la ejecución del
  ↳ circuito en tiempo real.
10 print(f">>> Job ID: {job.job_id()}")
11
12 # Consulta del estado de la ejecución del
  ↳ circuito cuántico
13 print(f">>> Job Status: {job.status()}")
```

```
1 # Nueva verificación del estado de la ejecución
  ↳ del circuito cuántico
2 print(f">>> Job Status: {job.status()}")
```

En el bloque de código anterior se presentan las instrucciones encargadas de ejecutar el circuito cuántico trasquilado, utilizando un *sampler* (denominado `sampler2run`) y especificando su modo de ejecución, el cual puede ser `batch`, `session` o `backend`. En este caso, se emplea el modo `backend`, lo cual implica que el circuito será empaquetado y enviado como una tarea de ejecución al computador cuántico (o al simulador, como es este caso).

Los otros modos de ejecución disponibles son:

- **Batch:** permite ejecutar múltiples tareas en paralelo que no dependen entre sí.

- **Session:** prioriza la disponibilidad de la QPU para un usuario específico durante una sesión activa.

Para ejecutar el *sampler* definido, se utiliza la función `.run()`, a la cual se le pasa como argumento el circuito a ejecutar y el número de repeticiones (o *shots*). En este caso, se especifica un total de 1024. El resultado de la ejecución queda almacenado en la variable `job`.

Cada trabajo enviado a un backend (ya sea hardware real o un simulador de IBM) posee un identificador único asociado. Una vez enviado el `job` a la máquina remota, resulta útil conocer el estado de la transacción. Para ello, se puede utilizar el método `.status()`, que permite verificar si el trabajo está, por ejemplo, en estado `QUEUED` (en cola de espera) o `DONE` (ejecutado exitosamente). Cuando el estado sea `DONE`, se puede proceder con la ejecución de los bloques de código siguientes.

```
1 result = job.result()
2
3 # Get results for the first (and only) PUB
4 pub_result = result[0]
5
6 # Get count measurement results
7 counts = pub_result.data.Medida.get_counts()
8
9 print(
10     f"Counts for the 'meas' output register:
11     ↳ {counts}")
12 )
```

Tras ejecutar el circuito en el backend, se procede a consultar el resultado de la tarea. Este se almacena en la variable `result`, utilizando el método `.result()` aplicado sobre el objeto `job` previamente generado.

Al indexar `result[0]`, se accede al primer (y en este caso único) *PUB* (*Primitive Unified Block*), es decir, al único bloque de primitivas que compone este circuito cuántico. A partir de este resultado, se puede obtener la cantidad de colapsos en los estados $|0\rangle$ y $|1\rangle$ mediante el método `.get_counts()`.

```
1 # Probabilidades de medición de cada salida, dada
  ↳ la ejecución real realizada.
2 plot_distribution(counts)
3 plt.show()
```

De esta variable se extrae la información de interés. En este caso, se desea conocer la distribución de probabilidad del vector de estado del circuito planteado; es decir, un histograma que contabilice, de los 1024 experimentos realizados, cuántos resultaron con un colapso en $|0\rangle$ y cuántos en $|1\rangle$.

Esta medida se almacena en la variable `counts`. Luego, se hace uso de la función `plot_distribution(counts)` para visualizar dicho histograma de forma rápida y sencilla.

```
1 prob0qc = counts['0']/shotno
2 prob1qc = counts['1']/shotno
3 #print("Observed probabilities of measuring the
  ↳ computational basis states",
  ↳ round(prob0qc,3), round(prob1qc,3))
4
5 #print("Statevector: ", out)
6 #print("Statevector probabilities of measuring the
  ↳ basis states: ", out.probabilities())
```

```

7 prob0statevec =
  ↳ out_statevector.probabilities()[0]
8 prob1statevec =
  ↳ out_statevector.probabilities()[1]
9
10 # one simple approach to calculate a quantum
  ↳ computer-modified complex number #
  ↳ amplitude^2 = probability
11 z0qc = z0*sqrt(prob0qc/prob0statevec)
12 z1qc = z1*sqrt(prob1qc/prob1statevec)
13 zqc = z0qc/z1qc
14 #zqcminus = -z0qc/z1qc
15 #print(z, zqc, zqcminus)
16
17 # compare the simulator state vector with
  ↳ the-running-on-a-real-quantum-computer
  ↳ modified state vector
18 print(z, zqc)

```

El bloque de código anterior representa el núcleo del *note-book*. En él, se utiliza la información cuántica obtenida a partir del simulador no ideal para generar el fractal. El procedimiento es análogo al realizado previamente para calcular el conjunto de Julia, aunque existen diferencias importantes.

La primera diferencia radica en que, mientras que el resultado consiste en un conteo de colapsos en los estados $|0\rangle$ y $|1\rangle$ cuya suma total es 1024, es necesario convertir estos valores en probabilidades. Para ello, tanto el número de ocurrencias de $|0\rangle$ como de $|1\rangle$ se divide por la cantidad total de ejecuciones (*shots*), es decir, 1024. Esta conversión a frecuencias relativas no fue necesaria en el caso del simulador Aer, ya que este entregaba directamente las probabilidades.

Las probabilidades obtenidas mediante el simulador Aer se almacenan en las variables `prob0statevec` y `prob1statevec`, y son utilizadas como referencia teórica para efectos de comparación.

Posteriormente, se procede con la construcción del número complejo semilla z_{qc} (análogo al z utilizado con el simulador ideal), calculando previamente los coeficientes $z0_{qc}$ y $z1_{qc}$.

Finalmente, se imprimen ambos valores complejos —el correspondiente al simulador ideal y el obtenido con el modelo ruidoso— para facilitar su comparación.

```

1 # Plot both Julia set fractals for comparison -
  ↳ the simulator based on the left and the
  ↳ modified on the right
2 f, axarr = plt.subplots(1,2,figsize=(12, 12))
3 axarr[0].imshow(julia_set(c=z), cmap='magma')
4 axarr[1].imshow(julia_set(c=zqc), cmap='magma')

```

Finalmente, para generar el fractal, se vuelve a ejecutar el algoritmo `julia_set`. En este caso particular, se realiza la ejecución dos veces: una para visualizar el fractal resultante del simulador ideal y otra para visualizar el fractal generado a partir del simulador con el modelo de ruido del hardware.

Ambas visualizaciones se presentan una al lado de la otra, lo que permite una comparación directa entre los resultados obtenidos en un entorno ideal y en uno ruidoso.

```

1 from qiskit import __version__
2 print(__version__)

```

Finalmente, se emplea una función para obtener la versión de qiskit instalada en el entorno de trabajo.

B. Conceptualización de la superposición e interferencia cuántica a raíz del algoritmo

Cuando se habla de *superposición cuántica*, se hace referencia a la posibilidad de que, mientras un sistema cuántico no haya colapsado a un estado medido, pueda encontrarse en una combinación de varios estados con cierta distribución de probabilidad. En sí mismo, el término “estado cuántico” está intrínsecamente ligado a una definición probabilística, no determinista, del qubit. A la luz del circuito cuántico utilizado en este informe, dicha superposición se induce mediante la implementación de una compuerta Hadamard, la cual por sí sola hace que el qubit se encuentre en una superposición equiprobable entre los estados $|0\rangle$ y $|1\rangle$.

Siendo más rigurosos, la compuerta Hadamard introduce un cambio en la base del sistema, pasando del dominio $|0\rangle$ y $|1\rangle$ al dominio $|+\rangle$ y $|-\rangle$, lo cual puede visualizarse como un giro en los ejes de referencia de la esfera de Bloch.

Además del efecto difusor de la compuerta H , la inclusión de compuertas como la U —con múltiples rotaciones arbitrarias— hace que el estado del qubit se torne más complejo. Y complejo no solo en el sentido de mayor sofisticación en su orientación dentro de la esfera de Bloch, sino también en cuanto a su naturaleza matemática *compleja*, dado que el estado está representado por amplitudes con números complejos.

En tanto la ubicación del vector de estado implique funciones complejas (amplitudes y fases respecto a cada eje), se puede decir que el qubit está en superposición de múltiples posibilidades, algunas con mayor probabilidad que otras, dependiendo de su cercanía a los estados base en los que puede colapsar. Al producirse la medición (o colapso), dicha superposición desaparece.

En resumen, el circuito cuántico estudiado permite comprender la superposición cuántica como la capacidad de un qubit de encontrarse simultáneamente en los estados $|0\rangle$ y $|1\rangle$ (base computacional), $|+\rangle$ y $|-\rangle$ (base de Hadamard), o $|i\rangle$ y $|-i\rangle$ (base diagonal imaginaria), y que las probabilidades de colapsar a uno u otro dependerán de las rotaciones y ángulos aplicados mediante compuertas cuánticas.

Respecto a la *interferencia cuántica*, esta resulta especialmente interesante al observar la interacción entre el estado cuántico resultante del circuito y otro sistema complejo. Una analogía adecuada para abordar este fenómeno es la interferencia óptica en experimentos con rendijas: las interferencias constructivas y destructivas entre ondas generan patrones cuya forma (como la periodicidad) depende de las características individuales de cada onda —como su frecuencia, fase y forma—.

En el contexto de este experimento, el sistema cuántico genera un vector de estado (de naturaleza matemática compleja), el cual interactúa con un entorno también complejo: el plano complejo definido para construir el fractal. Esta interacción conduce a que ciertos puntos del plano diverjan o converjan a diferentes ritmos durante el proceso iterativo.

Lo cuántico en este caso es la *semilla*, es decir, la constante c que inicializa todos los valores del plano complejo z , y

que es obtenida a partir del cociente entre las amplitudes del estado cuántico final del circuito. Esa semilla, al contener una información de fase y magnitud originada en la superposición y rotaciones cuánticas del circuito, puede interferir de forma constructiva o destructiva con los valores de z durante la generación del conjunto de Julia. Aunque el plano complejo como tal no proviene de un sistema cuántico, sí puede considerarse una representación análoga a otro sistema en el que la interacción con el estado cuántico tiene un impacto observable —en este caso, en la forma y nivel de contraste del fractal generado.

C. Cómo se crea un fractal del Conjunto de Julia

Como se explicó en el punto anterior, el génesis de los patrones visualizados radica en la *interferencia cuántica* entre dos sistemas complejos: el estado cuántico resultante del circuito y el plano complejo definido computacionalmente. Cada punto del plano complejo interactúa con la semilla c , constante compleja calculada a partir del cociente entre los coeficientes (amplitudes) del estado cuántico del circuito. A pesar de que todos los puntos comparten la misma semilla c , cada uno de ellos z exhibirá una evolución distinta a lo largo del tiempo iterativo.

El fractal generado corresponde a un *conjunto de Julia*, el cual se construye mediante la técnica de *escape temporal* (*escape-time fractals*). Este enfoque consiste en aplicar iterativamente la función

$$z_{n+1} = z_n^2 + c$$

sobre cada punto z del plano complejo. En este proceso, un punto puede o no “escapar” de un umbral definido —en este caso, un módulo mayor a 2— dentro de un número determinado de iteraciones. A cada punto se le asigna un color o intensidad en función del número de iteraciones que tarda en superar dicho umbral (denominado *divergence time* o *div_time*). Así, el color de cada píxel del fractal refleja cuán rápidamente diverge, teniendo como semilla el número complejo c derivado del estado cuántico.

Ahora bien, la visualización del conjunto de Julia se ve influenciada también por la calidad de la semilla cuántica. En el caso con el simulador `Aer`, se puede observar un fractal con destacable nitidez, claridad y contraste, debido a que la representación del estado cuántico está libre de ruido estadístico. No obstante, cuando se ejecuta en hardware, o en este caso, en un *backend* realista con modelo de ruido, como el `FakeBogotaV2`, se hace evidente la incidencia de errores asociados a un computador cuántico real, como la decoherencia, errores de compuerta o ruido en la medición. Esto hace que la interacción de la semilla con el plano complejo no sea tan clara.

Se podría hacer una analogía con un experimento con dos fuentes de luz o mediante el experimento de la doble rendija, donde un patrón de difracción puede ver comprometida su claridad, contraste o nitidez si alguna de las rendijas o fuentes puntuales de luz vibra o sufre algún tipo de ruido mecánico y

óptico, de tal manera que se observe un patrón de difracción difuso.

D. Generación de fractales a partir de una función no-lineal

Continuando con el desarrollo experimental, cabe preguntarse qué patrones de fractales resultan en el caso de que el estado cuántico, la semilla empleada para la generación del Conjunto de Julia, evolucione temporalmente de tal forma que ya no sea una constante. Para esto, se realizaron modificaciones al código proporcionado. Dicha versión es accesible a través del siguiente repositorio de GitHub creado para complementar este trabajo: `introducci-n-a-Qiskit-fractales`. Algunos de los fractales generados, se presentan en las figuras 1 y 2.

Fractal de Julia con c generado aleatoriamente.

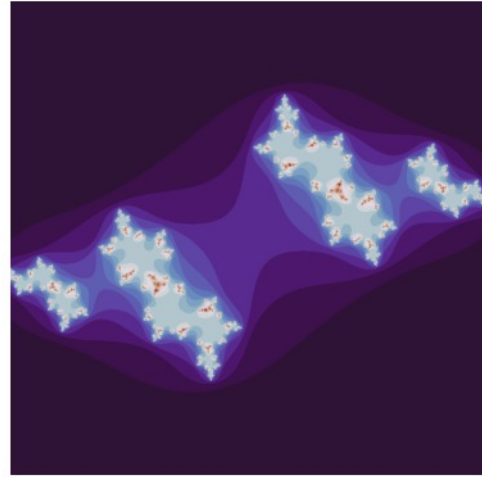


Fig. 1. Primer fractal obtenido al variar parámetros y compuertas cuánticas.

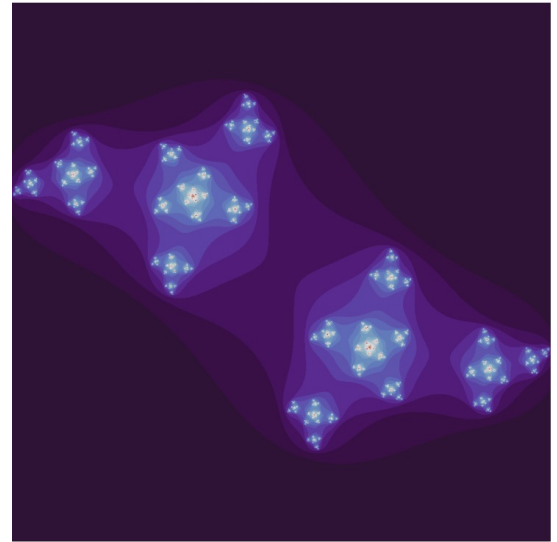


Fig. 2. Segundo fractal obtenido al variar parámetros y compuertas cuánticas.

IV. CONCLUSIONES

- A lo largo de este trabajo se diseñó e implementó una secuencia personalizada de compuertas cuánticas, construida a partir de operadores estándar como Hadamard, Pauli-X, compuerta S y una unidad unitaria general $U(\theta, \phi, \lambda)$. Esta combinación dio lugar a una operación compuesta sin equivalente directo en el conjunto canónico de compuertas cuánticas, permitiendo observar comportamientos particulares del sistema al aplicarse repetidamente.

La experimentación con este tipo de estructuras evidencia la flexibilidad del paradigma cuántico para construir operaciones complejas a partir de bloques básicos. Además, destaca el potencial de personalizar compuertas para aplicaciones específicas, como la simulación de sistemas dinámicos, algoritmos iterativos, o incluso patrones con comportamiento fractal. Esta aproximación fomenta la creatividad en el diseño de circuitos cuánticos y sienta las bases para la exploración de nuevas estrategias en el desarrollo de algoritmos en computación cuántica.

- A partir de los resultados obtenidos con el código personalizado a partir del código explicado en este documento, se puede llegar a inferir que, la periodicidad espacial con la que se presentan patrones en el plano complejo puede variar dependiendo no solo dependiendo del estado cuántico del circuito ejecutado, sino también de si varía o no la semilla C con cada iteración. A raíz de lo evidenciado con el experimento, al hacer variar C los patrones parecen más distanciados entre si, lo que podría indicar que la variabilidad de c hace que las condiciones para que se produzca la divergencia de los z sean más difíciles de lograr (dificultad para encontrar momentos de máxima interferencia constructiva o destructiva). Esto podría tener aplicaciones útiles en el campo de la criptografía, donde semillas de encriptación variables podrían dificultar las labores de desencriptación por parte de terceros.
- Tras la ejecución del circuito en un Backend de NISQ como el FakeBogota, se hace evidente que el ruido es aún una materia de suma importancia en el uso del Hardware cuántico. No ser conciente de esta problemática puede inducir a errores en interpretaciones experimentales y a un gran desperdicio de recursos de Hardware que, de no ser optimizados, encarecerían y limitarían el acceso a estas tecnologías. De esto, es más que claro por qué el ruido y la decoherencia son problemas que, incluso desde el ambiente entusiasta, se intentan solucionar con múltiples técnicas. A raíz de esto, se puede ser optimista respecto a que tecnologías como los qubits topológicos puedan ayudar a que estas tecnologías sean consideradas una apuesta más segura para ciertos sectores de la industria que no pueden permitirse ya sea elevados costos o resultados difusos.
- Algo interesante es que los sistemas dinámicos no lineales, combinados con parámetros generados por qubits, son muy sensibles a pequeños cambios. Si se cambia un

valor solo un poco, el fractal que se genera puede ser completamente diferente. Esto se parece a lo que pasa en la criptografía: si alguien cambia un mensaje o una clave solo un poco, el resultado final también cambia por completo. Por eso, tal vez estas propiedades se podrían usar para transmitir información de forma muy segura, porque cualquier interferencia en el proceso se notaría fácilmente, porque tiene un efecto grande

- También se puede ver el fractal como una huella digital, una imagen única que viene de un estado cuántico. Aunque z sea aleatorio, el resultado es siempre diferente. Esto podría usarse como una clave segura o para identificar a alguien. También podría servir para saber si un circuito cuántico está funcionando bien, usar para proteger información o crear firmas digitales especiales. Nótese cómo, con el paso del mundo clásico al mundo cuántico en la computación, existe un inherente paso del mundo de la pseudoaleatoriedad, al mundo de una aleatoriedad real, y que existe gracias al continuo en el que existe el qubit previo a su colapso.
- En la naturaleza también existen fractales, como el brócoli, las venas o un río que fluye hacia el mar. Aunque no sean fractales del tipo Julia Set, tal vez se podrían identificar y clasificar ciertos "fractales naturales" usando estados cuánticos. También se podría intentar imitar estas formas y encontrar nuevas morfologías, parecidas a las que ya existen en la naturaleza. Esto podría servir, por ejemplo, para descubrir nuevos materiales o estructuras útiles.

También, desde la medicina, se podría modelar el crecimiento de tumores mediante como si fueran fractales. Dicho esto, se podrían entrenar modelos de inteligencia artificial para analizar tumores. Crear fractales artificiales parecidos a fragmentos reales de tumores, y así generar muchos más datos para el entrenamiento.

REFERENCES

- [1] W. Mazin, "Creating Fractal Art With Qiskit," *Qiskit Medium*, Jun. 1, 2022. [Online]. Available: <https://medium.com/qiskit/creating-fractal-art-with-qiskit-df69427026a0>
- [2] C. McGinty, "White Paper: A New Classification of Quantum-Fractal Alloys: Identification and Analysis of Nyrrite," *LinkedIn*, Oct. 27, 2024. [Online]. Available: <https://www.linkedin.com/pulse/white-paper-new-classification-quantum-fractal-alloys-chris-mcginty-equsc/>
- [3] 3Blue1Brown, "Beyond the Mandelbrot set, an intro to holomorphic dynamics," *YouTube*, Oct. 16, 2021. [Online Video]. Available: <https://www.youtube.com/watch?v=LqbZpur38nw>
- [4] Drojassa, "FRACTALES Y FRACTALES DE JULIA (EXPLICACIÓN BREVE)," *YouTube*, Oct. 5, 2020. [Online Video]. Available: <https://www.youtube.com/watch?v=HByye51O1IM>
- [5] IBM Quantum, "Qiskit SDK Documentation (v2.0.0)," [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit>
- [6] IBM Quantum, "Qiskit Aer Simulator Documentation," [Online]. Available: https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html
- [7] J. Cepelewicz, "The Quest to Decode the Mandelbrot Set, Math's Famed Fractal," *Quanta Magazine*, Jan. 26, 2024. [Online]. Available: <https://www.quantamagazine.org/the-quest-to-decode-the-mandelbrot-set-maths-famed-fractal-20240126/>
- [8] I. A. Mageed, "Fractal Open Problems in Cancer Research, Medicine, Biomedicine, Clinical Sciences, and Dentistry," *Journal of Clinical Bio

Medical Advances*, vol. 4, no. 1, pp. 01–09, 2025. [Online]. Available:
https://mkscienceset.com/articles_file/367-_article1738404644.pdf